

Searching Binary Embeddings

Calculations and Experiments

12 September 2026

Abstract

This report compares clustered scanning, Bitplanes and Backward Walk on one collection of binary documents. The calculations explain how cluster size, bit splits and key lookups affect work and memory. Experiments check those estimates and measure when each method is faster under the same recall and RAM requirement.

1 Introduction

1.1 Exa method and end to end pipeline

Exa describes a search pipeline that shortens Matryoshka embeddings, stores document values as binary signs, selects nearby clusters, and scores their documents with the floating-point query [1]. The results can then be reranked with uncompressed vectors. We refer to that approach as the original method, or A. The experiments use our implementation of this search stage; they do not measure Exa’s production system.

Documents are indexed before queries arrive. At query time, A opens some clusters and scores their documents. The blue box is the step that B and C change.

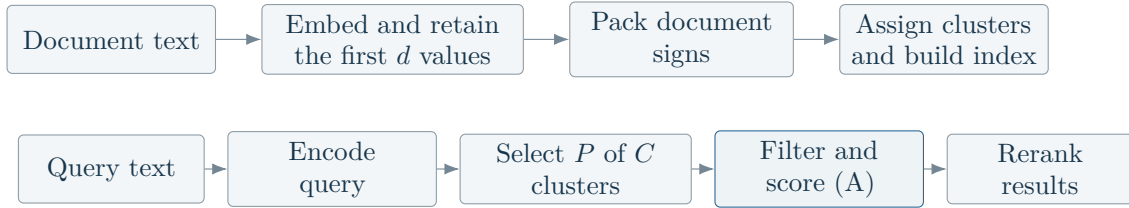


Figure 1: Exa’s described end-to-end flow. B and C replace the blue search step. The benchmark starts from cached embeddings and times cluster selection and search.

1.2 Idea background

The query itself tells us the best possible combination of document signs. To follow the arithmetic, consider a five-value example:

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

A document bit of 1 represents +1; a bit of 0 represents −1. At the first position, $0.7 \times (+1) = 0.7$ is better than $0.7 \times (-1) = -0.7$. At the third position, $-0.4 \times (-1) = 0.4$ is better than $-0.4 \times (+1) = -0.4$. Following this rule at every position gives 11001, with score $0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5$.

A mismatch reduces the score by twice the size of that query value. Mismatching the first position loses 1.4; mismatching the last loses 0.6. In general,

$$S(q, b) = Q - 2p, \quad Q = \sum_i |q_i|, \quad p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|. \quad (1)$$

Here p adds the sizes of the query values at mismatched positions. For example, 11000 differs only at the last bit, giving score $2.5 - 2(0.3) = 1.9$.

The question is whether an index can find documents near the preferred pattern without scoring every document. Bitplanes use the query values to choose which bits to split on. Backward walk starts with a short preferred pattern and tries other patterns when needed. The preferred pattern may not exist in the collection, so both methods must retain alternatives.

1.3 Exploring two new methods: “Bitplanes” and “Backward Walk”

1.3.1 Bitplanes

Method B keeps the packed rows and adds a second arrangement of their signs. Each *bitplane* stores one bit position across all documents in a cluster. The figure shows three documents with the query from Section 1.2. A candidate mask records which local rows remain.

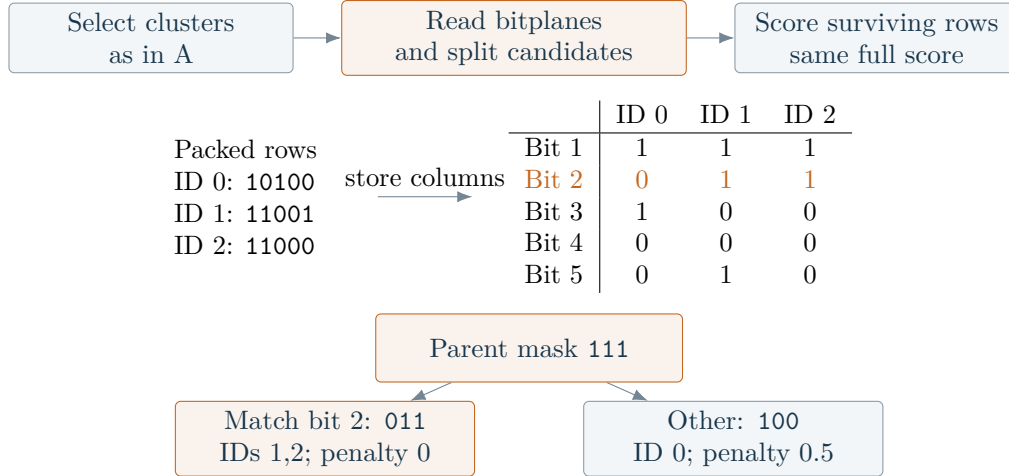


Figure 2: Orange marks the changed stage and the selected plane. Both child masks retain all three positions. On a larger cluster, this physical width is a major cost.

The absolute sizes of the query values give this position order: 1, 4, 2, 3, 5. For a positive query value, a parent mask A is split into $A \& B_i$ and $A \& \neg B_i$. The first child matches; the other adds $|q_i|$ to its penalty. Negative query values reverse the roles. Alternatives remain in a priority queue, ordered by known penalty, because a preferred early bit does not determine the final ranking.

Equation 1 gives a safe upper bound: a branch with known penalty p can score at most $Q - 2p$. If IDs 1 and 2 score 2.5 and 1.9, the alternative with penalty 0.5 has bound 1.5 and cannot improve the top two. Equal-score cases retain the common document-ID tie rule.

```

bitplanes(query, clusters, K, leaf_size, node_budget):
    table, best = prepare_score_table(query), empty_top_k(K)
    order = positions_by_decreasing_abs_value(query)
    queue = full candidate mask for each selected cluster
    while queue is not empty:
        node = remove_lowest_penalty(queue)
        if cannot_improve(node, query, best): continue
        if node_budget_is_reached(): break
        if node.count <= leaf_size or no_bits_remain(node):
            score_set_documents(table, node.mask, best)
        else:
            queue_nonempty_children(split(node, order))
    return best.sorted_by_score_then_id()

```

The leaf size decides when to stop splitting and score the remaining rows. Unlimited search with safe bounds is exact within opened clusters; a finite work limit can lose results.

1.3.2 Backward Walk

Method C forms a short key from a chosen set of h bit positions. In each cluster, documents with the same key are stored consecutively. A directory maps an occupied key to the start and length of that row range. Search begins at the query's preferred key and enumerates alternatives in increasing weighted mismatch penalty.

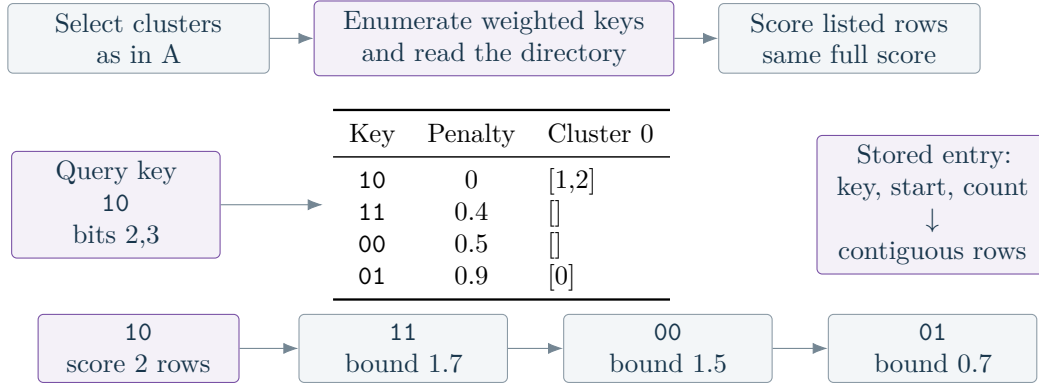


Figure 3: Purple marks the changed stage. Here the first key returns IDs 1,2. Their scores are 2.5 and 1.9, so the next key's bound 1.7 already permits stopping. The remaining entries show the order that a query requiring more work would follow.

The ordering uses the sum of absolute query values at the flipped positions. Counting flipped bits alone would give the wrong order when query weights differ. There is one shared enumeration queue across opened clusters, followed by a lookup in each cluster's directory. Empty lookups also take time. Removing a key constraint is not a single ordinary hash lookup: the newly admitted patterns must be enumerated or supported by a different data structure.

```
backward_walk(query, clusters, K, key_positions, limits):
    table, best = prepare_score_table(query), empty_top_k(K)
    keys = unique_keys_by_increasing_penalty(query, key_positions)
    for key, penalty in keys:
        if cannot_improve(penalty, query, best): break
        for cluster in clusters:
            if work_limit_reached(limits):
                return best.sorted_by_score_then_id()
            rows = cluster.directory.lookup(key)
            score_each_row(table, rows, best)
    return best.sorted_by_score_then_id()
```

A short key omits some bit positions, so every retrieved row still receives the complete score. Finding enough candidates does not by itself prove sufficient recall. Exhausting the keys or ruling out every remaining bound is exact within opened clusters; stopping at a candidate or lookup budget is approximate. Neither B nor C can recover a document excluded by the initial routing.

2 Preliminary calculations

The collection contains $N = 1,000,000$ documents. The same 200 query vectors, 256-bit codes and reference answers are used throughout. Each method may choose its own cluster count C , probe count P and local settings. A probe opens one cluster. The result count is $K = 100$; d denotes the number of document bits.

2.1 Time complexity

One example for all three methods. Open one cluster containing these three rows. Return the best two results: $N = 3$, $C = P = 1$, $d = 5$, $K = 2$.

$$q = [0.7, 0.5, -0.4, -0.6, 0.3], \quad \text{ideal signs: 11001.}$$

ID	Document bits	Full score	Calculation
0	10100	0.1	$0.7 - 0.5 - 0.4 + 0.6 - 0.3$
1	11001	2.5	$0.7 + 0.5 + 0.4 + 0.6 + 0.3$
2	11000	1.9	$0.7 + 0.5 + 0.4 + 0.6 - 0.3$

- These are teaching rows, not a change to the benchmark dataset.
- Bits are shown in position order, left to right. Scores are rounded here.
- For larger collections, $n = N/C$ estimates rows per cluster if clusters are equal-sized.

2.1.1 Original method

Step 1: open the cluster and prepare the query.

- Open cluster 0: [ID 0, ID 1, ID 2]. One cluster lookup here.
- Following Exa's four-sign table method [1], split the five query values into $[0.7, 0.5, -0.4, -0.6]$ and $[0.3, 0, 0, 0]$.
- Each group has 16 sign patterns: $2 \times 16 = 32$ table entries. The final group is padded.
- The code checks four slots per entry: $32 \times 4 = 128$ slot visits. Table setup is $O(d)$ for a fixed group width of four. Cluster selection costs T_{route} .

Step 2: score each document.

$$\text{ID 0} \longrightarrow 0.1 \quad \text{ID 1} \longrightarrow 2.5 \quad \text{ID 2} \longrightarrow 1.9$$

- Loop over three rows; each reads two score-table entries: $3 \times 2 = 6$ terms.
- For F_A rows, this is $F_A \lceil d/4 \rceil$ terms, or $O(F_A d)$ work.

Step 3: keep the best two.

$$[0:0.1] \longrightarrow [1:2.5, 0:0.1] \longrightarrow \boxed{[1:2.5, 2:1.9]}.$$

- Offer three scores; keep at most two. The display above is sorted for readability.
- Keeping K results costs up to $O(F_A \log K)$; final sorting costs $O(K \log K)$.

Total. One cluster lookup, 32 table entries built, six table terms read and three scores offered. These are different kinds of work, so we do not add them as equal CPU operations.

$$T_A \approx T_0 + \frac{PN}{C} a_A.$$

T_0 includes cluster selection and table setup. a_A is average time per document, including its score and result update. For fixed C, P, d, K , scan work grows as $O(N)$.

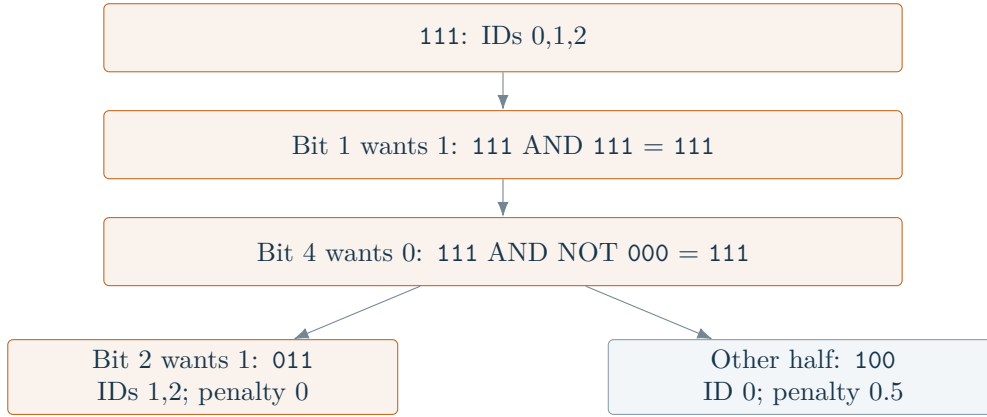
2.1.2 Bitplanes

Same input: $q = [0.7, 0.5, -0.4, -0.6, 0.3]$; rows 10100, 11001, 11000; best two results. Stop splitting when at most two rows remain. Keep all other work limits unlimited.

Step 1: open, prepare and order the bits.

- Open the same cluster and build the same 32 score-table entries as A.
- Sort five query positions by absolute value: 1 (0.7) $\rightarrow$ 4 (0.6) $\rightarrow$ 2 (0.5) $\rightarrow$ 3 (0.4) $\rightarrow$ 5 (0.3).
- Sorting costs $O(d \log d)$. One pass sums the five absolute values: $Q = 2.5$, costing $O(d)$.
- Write one word for mask 111: all rows are active. Initial masks cost $O(PW)$ word writes.

Step 2: split the mask.



- The first two splits remove nothing. The third separates ID 0 from IDs 1 and 2.
- All three masks still have three positions. Each occupies one 64-bit word.
- Three splits visit $3 \times 1 = 3$ word positions. With J splits and mask width W , this costs $O(JW)$, where $W = \lceil n/64 \rceil$.

Step 3: score the two remaining rows.

- Mask 011 fits the leaf limit. Read one mask word, then score IDs 1 and 2.
- Two scores use $2 \times 2 = 4$ table terms. Best results: [1: 2.5, 2: 1.9].
- Reading L final masks costs $O(LW)$; scoring F_B rows costs $O(F_B d)$. Result updates have the same $O(F_B \log K)$ bound as A.

Step 4: check the waiting branch and stop.

- Its best possible score is $2.5 - 2(0.5) = 1.5$, below the retained score 1.9.
- Skip ID 0. One final bound check is $O(1)$ once the query sum and penalty are known.
- Four candidate sets were processed: three splits and one leaf. Including the P initial sets, queue work is bounded by $O((P + V) \log(P + V + 1))$.

Total. Same table setup as A; sum five weights; sort five positions; write one initial mask; visit three split words and one final-mask word; score two rows; skip one row. Fewer scores alone do not prove lower time.

$$T_B \approx T_0 + PS \left\lceil \frac{N}{64C} \right\rceil b + f_B \frac{PN}{C} a_B + T_{\text{other},B}.$$

S is average splits per opened cluster; f_B is the scored fraction; b is time per split-word visit. Here $S = 3$, $f_B = 2/3$. Other time includes sorting, queues and final-mask reads. A wide mask still costs time after many rows are removed.

2.1.3 Backward Walk

Same input: $q = [0.7, 0.5, -0.4, -0.6, 0.3]$; rows 10100, 11001, 11000; best two results. Use bits 2 and 3 as the key: $h = 2$. Keep lookup and candidate limits unlimited.

Step 1: form the preferred key.

$$q_2 = +0.5 \rightarrow 1, \quad q_3 = -0.4 \rightarrow 0, \quad \boxed{\text{preferred key} = 10}.$$

- Open the same cluster and build the same 32 score-table entries as A.
- Read two signs: $O(h)$. Sort the two weights: $O(h \log h)$.
- One pass sums all five absolute query values: $Q = 2.5$, costing $O(d)$.
- Flipping bit 3 costs 0.4 in penalty; flipping bit 2 costs 0.5. Try the smaller penalty first.

Step 2: look up the preferred key.



- One lookup finds two rows. The other stored key, 01, contains ID 0.
- Score both rows with all five query values: four table terms, not just two key bits.
- H hash lookups cost expected $O(H)$ under normal hash-table behavior. Scoring costs $O(F_C d)$, plus $O(F_C \log K)$ result updates.

Step 3: check the next key and stop.

$$\underbrace{10}_{\text{score IDs 1,2}} \longrightarrow \underbrace{11}_{\text{penalty 0.4, bound } 2.5-0.8=1.7} \longrightarrow \boxed{1.7 < 1.9 : \text{stop}}.$$

- The next key cannot beat the current second result. Do not look it up.
- The remaining keys have larger penalties, so they cannot improve the result either.
- One key was looked up; one next-key state was queued. The final bound check is $O(1)$.
Generating and ordering E visited keys costs up to $O(E \log(E + 1))$ after the initial sort.

Total. Same table setup as A; sum five weights; sort two key positions; make one lookup; score two rows; check one next-key bound. All methods return IDs 1 and 2, so this example has 100% recall. It does not establish a speedup on real data.

$$\boxed{T_C \approx T_0 + P E c + F_C a_C + T_{\text{other}, C}.$$

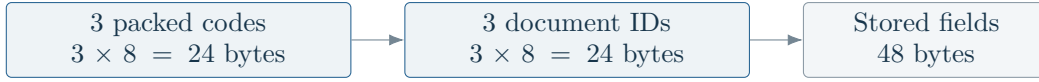
- E is keys tried in each opened cluster; c is time per lookup; F_C is rows scored. Other time includes key ordering and range starts. Use actual lookup count if a budget stops mid-key.
- Only for equal rows per key, $F_C \approx \text{PEN}/(C 2^h)$. Here that estimate gives 3/4 of a row, while the actual key holds two. Count the stored rows for an exact answer.
- Wider keys can mean fewer rows per hit but more empty lookups. The required recall sets how far the search must continue.

2.2 Memory complexity

Same example: three documents, five bits each, one cluster, two results. We count bytes of data first. Full process RAM also includes unused capacity, queue records, hash-table overhead, Python, and other runtime memory.

2.2.1 Original method

Step 1: store the codes and IDs before search.



- Each five-bit row uses one eight-byte word. Packing all 15 bits across row boundaries would use a different layout; this implementation stores whole words per row.
- Formula: $M_{\text{rows}} = N(8 \lceil d/64 \rceil + 8)$ bytes. Size is $O(N \lceil d/64 \rceil + N)$.
- Cluster data is extra. For example, C float32 centroids of width d_R use $4Cd_R$ bytes. This one-cluster example needs no centroid search.

Step 2: hold the query, score table and current results during search.

$$\boxed{5 \times 4 = 20} + \boxed{32 \times 8 = 256} + \boxed{2 \times (8 + 8) = 32} = 308 \text{ bytes.}$$

- Left to right: float32 query, double-precision table entries, result IDs and scores.
- The result buffer holds two entries even though A checks three documents. It does not need a stored score for every document.
- General sizes: $4d + 128 \lceil d/4 \rceil + 16K$ bytes, or $O(d + K)$ for these buffers.

Total counted fields: 48 bytes stored + 308 query bytes = 356 bytes. This is not total process RAM; cluster records and allocation overhead remain additional.

2.2.2 Bitplanes

Step 1: keep A's rows and add the bitplanes.

	ID 0	ID 1	ID 2	Packed storage
Bit 1	1	1	1	8 bytes
Bit 2	0	1	1	8 bytes
Bit 3	1	0	0	8 bytes
Bit 4	0	0	0	8 bytes
Bit 5	0	1	0	8 bytes

- Each plane has three useful bits but occupies one word. Five planes add $5 \times 8 = 40$ bytes.
- Stored fields: $48 + 40 = 88$ bytes.
- Formula: $M_{\text{planes}} = 8d \sum_c \lceil n_c/64 \rceil$ bytes. Include padding in every cluster.

Step 2: hold masks while splitting.

$$\underbrace{111}_{\text{parent: 8 bytes}} \longrightarrow \underbrace{011}_{\text{child: 8 bytes}} + \underbrace{100}_{\text{child: 8 bytes}} .$$

- The parent exists while both children are built: $3 \times 8 = 24$ live mask bytes at the peak. All three masks keep their full width, even if one child is empty.
- Sorting five positions adds $5 \times 8 = 40$ bytes on this 64-bit machine.
- Counted query data: A's $308 + 24 + 40 = 372$ bytes. Queue records are still extra.
- For mask width $W_c = \lceil n_c/64 \rceil$ and $A_c(t)$ live masks, peak mask data is $8 \max_t \sum_c A_c(t) W_c$ bytes. Waiting branches count too.

Total counted fields: 88 bytes stored + 372 query bytes = 460 bytes. Stored size is $O(N \lceil d/64 \rceil + N + d \sum_c W_c)$; query masks grow with the number and width of live branches, not just the number of remaining document IDs.

2.2.3 Backward Walk**Step 1: group rows by their two-bit key before search.**

Key	Stored rows	Start	Count
10	[ID 1, ID 2]	0	2
01	[ID 0]	2	1

- Keep one copy of the codes and IDs: 48 bytes. Rows with the same key sit together.
- Each entry holds a four-byte key, eight-byte start and eight-byte count: 20 bytes of fields. Two occupied keys add $2 \times 20 = 40$ bytes. Empty keys have no stored entry.
- Stored fields: $48 + 40 = 88$ bytes. Formula: $M_{\text{entries}} = U(4 + 8 + 8)$ for U occupied keys. Total stored size is $O(N \lceil d/64 \rceil + N + U)$, plus the hash table's allocation overhead.

Step 2: hold the key order and next key during search.

$$\underbrace{[3, 2]}_{\text{two positions: 16 bytes}} \quad \underbrace{11, \text{ penalty } 0.4}_{\text{one waiting state: 16 bytes}} .$$

- The waiting state stores an eight-byte penalty, four-byte flip mask and four-byte position. This example's measured key-queue peak is 16 bytes.
- Counted query data: A's $308 + 16 + 16 = 340$ bytes. Selected-cluster pointers and runtime objects are extra. With Q waiting states, the extra arrays take $O(h + Q)$ space.

Total counted fields: 88 bytes stored + 340 query bytes = 428 bytes.

What these totals mean.

- The totals add the listed buffer sizes; their peaks need not occur together. They are not measured process RAM. Reserved space and unlisted objects use extra bytes.
- The 32 GB experiment limit applies to measured process RAM, including overhead.
- Storage is built once. Query buffers are temporary; concurrent queries need their own work space.

2.3 Recall estimation

- Reference IDs are the exact best results under the full binary score. In our small example they are $[1, 2]$. All methods return both: $2/2 = 100\%$ recall.

- In the benchmark, each query has $K = 100$ reference IDs. Returning 99 gives 99% recall. This checks the ranking, not whether the text answers a person’s question.

Let G_q be reference IDs and $O_{a,q}$ returned IDs. Across the same $n_q = 200$ queries,

$$\overline{R}_a = \frac{1}{n_q K} \sum_q |G_q \cap O_{a,q}|.$$

2.3.1 Original method

Let g_q reference IDs be in the opened clusters. A scores all opened rows, so recall is g_q/K .

2.3.2 Bitplanes

If B returns fraction $r_{B,q}$ of those IDs, recall is $(g_q/K)r_{B,q}$. For example, $(8/10)(6/8) = 60\%$. Safe, unlimited local search gives $r_{B,q} = 1$.

2.3.3 Backward Walk

Likewise, recall is $(g_q/K)r_{C,q}$. Visiting or safely ruling out all keys gives $r_{C,q} = 1$. Limits can lose results. If $g_q = 0$, recall is zero. Average the per-query products; multiplying separately averaged fractions can give a wrong answer.

2.4 Summary

Each formula uses its method’s own opened clusters and local work. Let M_0 be allocated row and router storage, Q_0 common query memory, D_C allocated directory memory, and Q_B, Q_C extra masks and queues. Allocation overhead is included in these memory terms.

Time complexity

Method	Approximate query time
Original method	$T_0 + (PN/C)a_A$
Bitplanes	$T_0 + PS \lceil N/(64C) \rceil b + f_B(PN/C)a_B + T_{\text{other,B}}$
Backward Walk	$T_0 + PEc + F_C a_C + T_{\text{other,C}}$; for equal rows per key, $F_C \approx PEN/(C2^h)$

The symbols and loop counts are defined in the steps above. Use actual cluster sizes and key counts for the measured comparison. Each cost belongs in one term only.

Memory complexity

Method	Stored index	Temporary query memory
Original method	M_0	Q_0
Bitplanes	$M_0 + M_{\text{planes,allocated}}$	$Q_0 + Q_B$
Backward Walk	$M_0 + D_C$	$Q_0 + Q_C$

The process, including these arrays and the running program, must fit 32 GB. A peak that includes construction is reported as such, not as index size.

Recall estimation

Method	Per-query recall	Local exactness condition
Original method	g_q/K	Score all opened documents
Bitplanes	$(g_q/K)r_{B,q}$	Complete search with safe bounds
Backward Walk	$(g_q/K)r_{C,q}$	Visit or safely rule out remaining keys

The reported batch value averages the same 200 reference answers.

3 Deriving Optima

The objective is minimum measured retrieval time at a required batch recall and a 32 GB RAM limit. Only index and search parameters change. Document codes, query vectors and reference answers remain identical. Each method chooses its own cluster count and local settings.

3.1 Original method

For a given layout, the reference answers show exactly how many clusters A must open. Suppose a reference document belongs to the fifth cluster in the query’s routing order. A includes it when $P \geq 5$. Record this rank for every reference document and query.

Let $s_{qi}^{(C)}$ be the cluster rank of reference result i for query q . At recall requirement ρ , the smallest qualifying probe count is

$$P_A(C, \rho) = \min \left\{ P : \frac{1}{n_q K} \sum_{q=1}^{n_q} \sum_{i=1}^K \mathbf{1}[s_{qi}^{(C)} \leq P] \geq \rho \right\}.$$

The indicator contributes one when the document’s cluster is opened. With 200 queries and 100 reference IDs each, 99% recall requires at least 19,800 recovered IDs across the batch. These ranks are calculated before timing the searches.

The experiment uses one global cluster and learned layouts with 256, 4096 and 8192 clusters. For each layout it computes probe cutoffs for 80%, 90%, 95% and 99% recall, and also opens all clusters. Query time and selected-row counts are then measured. These cutoffs describe this query batch; they do not guarantee the same recall on future queries.

A derivative explains the competing costs. Suppose clusters are balanced, routing costs αC , each row costs approximately β , and P stays constant. Then

$$T_A(C) \approx \alpha C + \beta NP/C, \quad \frac{dT_A}{dC} = \alpha - \frac{\beta NP}{C^2}, \quad C^* = \sqrt{\frac{\beta NP}{\alpha}}.$$

This balances routing more clusters against scanning fewer rows. Under a recall requirement, $P_A(C, \rho)$ can change with C . The measured comparison uses that probe count and actual cluster sizes; the fixed- P derivative is not a complete solution.

3.2 Bitplanes

Consider a mask covering 6400 documents, or 100 words. If 200 candidates remain and a split removes 100, it saves 100 complete scores. It also adds 100 split-word visits and branch work. Let $W = \lceil n/64 \rceil$ be the bitmap width. If an extra split avoids ΔF scores, a simple comparison is

$$\Delta F a_B > Wb + \Delta T_{\text{other},B}.$$

The left side is saved scoring time. The right side is split time plus the change in other branch work. The comparison assumes a roughly constant cost per score; actual timing also captures changes in memory access. The complete search must retain the required recall.

If each split halves one candidate set, after j cuts it contains approximately $n2^{-j}$ rows. Holding other branch work constant gives the guide

$$T_B(j) \approx T_0 + jWb + n2^{-j}a_B + T_{\text{other},B}, \quad j^* = \log_2 \left(\frac{n \ln(2) a_B}{Wb} \right).$$

This is an illustrative cost balance, not a recall estimate for these embeddings. If two bitplanes are identical, splitting on the second after the first removes no additional rows. Alternative branches also add paths. The experiment measures these effects rather than assuming repeated halving.

B uses every cluster and probe option above, then varies local search: leaf size 128 with node budgets 256 and 1024; leaf size 512 with unlimited work; and leaf size N , which permits scoring an opened cluster without splitting. A node budget counts visited candidate sets, including leaves. Leaf sizes 32 and 128 are also compared at budget 4096. Actual splits, final-mask reads and scored rows explain the resulting time.

The probe cutoff computed for A is a necessary routing condition for B using the same layout. It may not suffice when local limits lose more results. Returned IDs determine whether each complete setting qualifies.

3.3 Backward Walk

The key must be wide enough to rule out some patterns. For the first saved query, the ideal full score is 12.361. Even if all eight key bits mismatch, the remaining full-score upper bound is 11.373. The 100th reference score is only 9.994. Thus even the least promising eight-bit key cannot be discarded using this bound. All 200 queries have this problem at key widths 1, 4, and 8.

At the same routing settings, changing key width from h to h' helps when

$$\boxed{\text{saved scoring time} > \text{extra lookup time} + \text{extra key-management time.}}$$

For example, saving 3 ms of document scoring pays for 1 ms of additional key work. Saving only 0.5 ms does not. These are cost examples; the measured comparison below checks both time and recall on the unchanged inputs.

A smaller first range does not determine the answer. Search may need many additional keys, including empty ones, before finding enough high-scoring documents or proving that remaining keys cannot improve the result.

The experiment tries keys of 1, 4, 8, 16, 20, and 24 bits. For all 200 queries, the short 1-, 4-, and 8-bit keys cannot be discarded using the score bound, so their complete key spaces are searched. Longer keys are compared with lookup limits of 4096 and 65,536. The full 256-bit score remains unchanged. Directory memory is measured for each index, and returned IDs establish recall. No $N/2^h$ occupancy assumption is used to assign recall or declare a winning key width.

3.4 Comparison

Let Θ_a contain method a 's tested settings. Its final choice is

$$\theta_a^* = \underset{\theta \in \Theta_a}{\operatorname{argmin}} T_{a,\text{measured}}(\theta) \quad \text{subject to} \quad \overline{R}_a(\theta) \geq \rho, \quad M_{a,\text{process}}(\theta) \leq 32 \times 10^9 \text{ bytes.}$$

Method	Adjustable settings	Work that may be reduced
Original method	Cluster count C and probes P	Documents reaching the scorer
Bitplanes	C , P , leaf size and node budget	Scores, at the cost of mask and branch work
Backward Walk	C , P , key width h and lookup limit	Scores, at the cost of lookups and range starts

The methods are compared after each has selected its own qualifying settings. Results below the recall target remain visible, but cannot establish a speedup at that target. This finds the best measured choices within the tested ranges on the unchanged query batch. It does not prove that every possible index configuration has been exhausted.

4 Experiments

4.1 Setup

Every run uses the same one million MS MARCO document embeddings [3], the same 200 query embeddings, and the same reference answers. The embeddings were already produced with Nomic [4]. We do not regenerate them, change their values, shorten them, or select different queries for different methods.

The documents contain 256 sign bits; queries retain 256 floating-point values. The reference answer is the best 100 document IDs from a full scoring calculation. A recall of 99% means returning 99 of those same 100 IDs on average. All methods use that identical reference.

Only the index parameters listed in Section 3 change: clusters, probes, B’s leaf size and branch budget, and C’s key width and lookup limit. The complete score still uses all 256 bits.

For each cluster count, the code finds probe counts reaching 80%, 90%, 95%, and 99% recall with the original scan. It also includes opening every cluster. These probe choices are available to A, B and C. This avoids giving A an unnecessarily expensive probe count when comparing it with a more selective method.

We measured 256 parameter settings in total.

The machine is an Apple M5 Max with 128 GiB of physical memory. Each process has a 32 GB memory budget and uses one CPU thread. Every setting searches all 200 queries twice. The fastest successful settings are then repeated in three processes, still using those same queries. Reported time is the middle measurement, or median.

Retrieval time includes choosing clusters and searching their documents. It excludes building the index and generating embeddings. File hashes for the document, query and reference arrays are checked before and after the measurements.

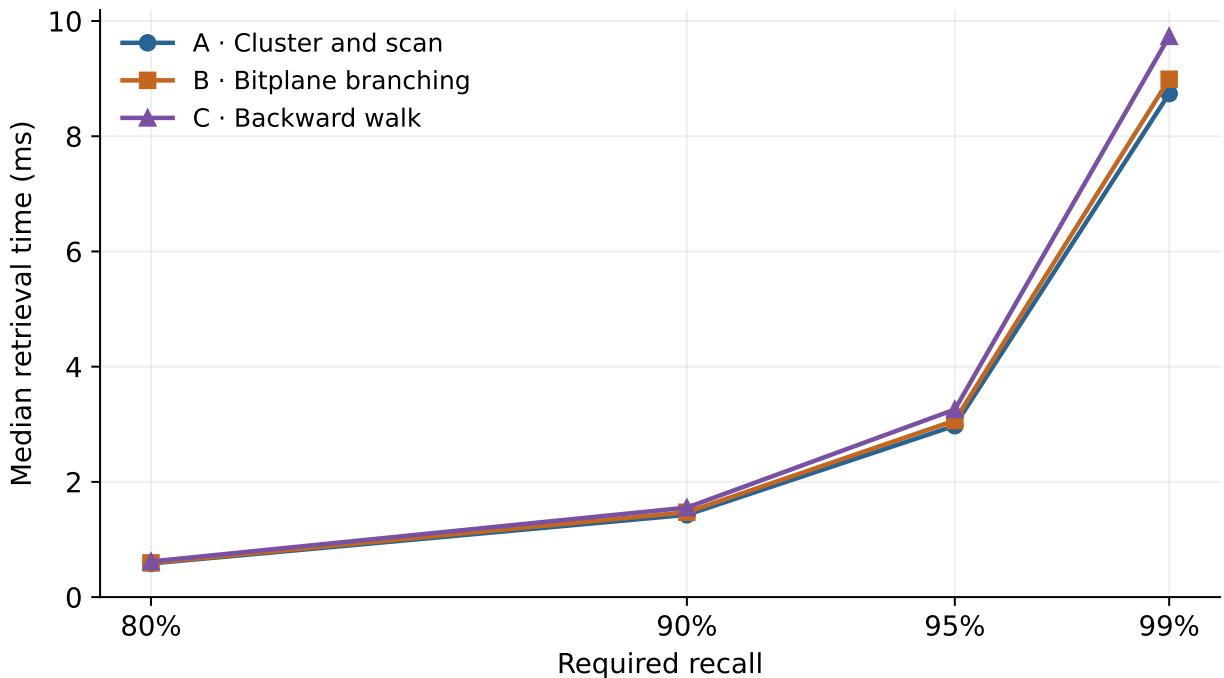


Figure 4: Time required at each recall target. All points use the same document and query arrays. Lower time is better; every plotted setting meets the recall requirement.

4.2 Results and calculation

Table 1: Repeated measurements at the same recall requirements.

Required recall	Method	Actual recall (%)	Median (ms)
80%	A	80.01	0.5842
80%	B	80.01	0.5936
80%	C	80.01	0.6181
90%	A	90.02	1.43
90%	B	90.02	1.47
90%	C	90.02	1.55
95%	A	95.00	2.97
95%	B	95.00	3.07
95%	C	95.00	3.26
99%	A	99.00	8.74
99%	B	99.00	8.99
99%	C	99.00	9.73

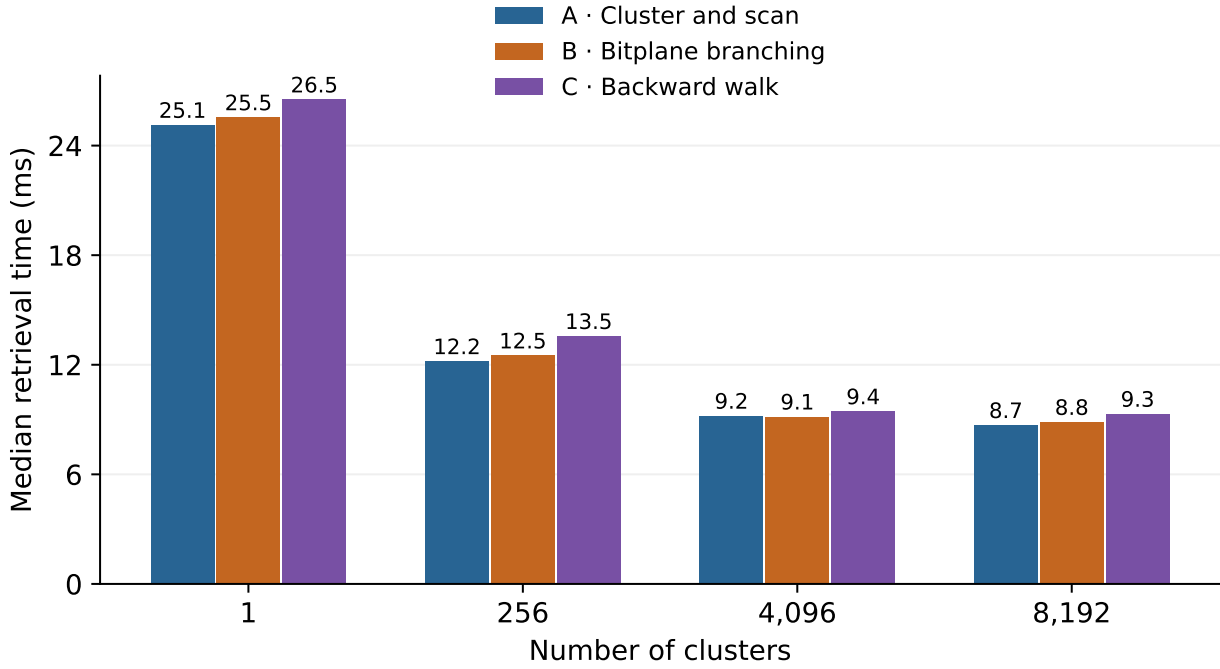


Figure 5: Initial measurements at each cluster count for 99% recall. Table 1 uses repeated runs of the selected settings, so small timing differences are expected. All inputs remain unchanged.

Table 2: Settings for 99% recall. L is leaf size and h is key width. Budget counts visited candidate groups for B and key lookups for C; zero means unlimited.

Method	Route	C	P	L	Budget	h	RAM (MB)
A	IVF	8,192	2,440	—	—	—	190.6
B	IVF	8,192	2,440	512	0	—	231.6
C	IVF	8,192	2,440	—	0	1	161.7

The following page connects those settings to their actual work. A name such as “Bitplanes” does not tell us whether the chosen setting split any candidates; a large leaf can make it score an opened cluster immediately.

4.3 What the numbers mean

Table 3: Average work per query for the selected 99% settings.

Method	Rows scored	Split words	Leaf words	Key lookups
A	295,858	0	0	0
B	295,858	0	5,820	0
C	295,858	0	0	4,880

At 99% recall, all three selected settings open 2440 of 8192 clusters and score the same 295,858 documents per query on average. B performs no splits and checks 5820 mask words. C uses a one-bit key and makes $2440 \times 2 = 4880$ lookups. Their median times are 8.74, 8.99 and 9.73 ms for A, B and C respectively. The extra index work avoids no full scores at these successful settings.

The smaller leaves and larger branch budgets were also tried. They did not produce a faster setting that met the same recall requirement. Wider C keys reduced some work but introduced another tradeoff: the best 24-bit result reached only 73.8% recall within the lookup limits tested. A 20-bit setting reached 95.315%, but took 29.71 ms, compared with 2.97 ms for A at the 95% requirement.

The code checks stored index data against the memory calculation: packed codes and IDs, padded bitplanes for B, and occupied directory entries for C. The measured process memory also includes allocation capacity, temporary arrays and the running program.

Table 4: Index data calculated from the layout and reported by the implementation.

Method	Calculated (MB)	Index data (MB)	Peak RAM (MB)
A	48.454	48.454	190.6
B	88.765	88.765	231.6
C	48.762	48.762	161.7

The parameter sweep answers which tested settings were fastest on this unchanged dataset. For A, increasing the recall requirement from 90% to 99% increases the opened clusters from 353 to 2440, out of 8192. Its median time rises from 1.43 to 8.74 ms. The query vectors stay the same; the extra work comes from opening more clusters. These results concern this dataset and the tested parameter ranges.

4.4 Code references

Run `experiments.fixed_data_comparison` with its `sweep`, `extend` and `repeat` commands. All three read the same arrays; the README gives the full commands and input paths. The search implementations are `cpp/index.cpp` and `cpp/key_index.cpp`. Results, parameter choices, input hashes and raw observations are saved under `results/fixed-data-comparison-2026-09-12/`.

5 Limitations and Next steps

This experiment compares the declared index settings on one unchanged document collection and query set. All methods use the same score, reference answers, RAM budget and CPU thread count. The selected settings are the fastest observed successful choices in that comparison; they are not a proof of the best possible implementation.

The formulas count document scores, mask reads and key lookups. Their time cost depends on memory access, code organization and the machine. Knowing the ideal binary pattern does not by itself tell us how many branches or keys must be visited to recover the reference answers. Those data-dependent counts must be checked rather than assumed.

A useful next step is to expand the parameter range or improve a demonstrated expensive operation while retaining this same benchmark. Any later comparison using a different embedding model, document collection or query set should be reported separately.

6 Conclusion

After allowing each method to choose its own cluster count and search settings, the original scan had the lowest measured time at the 80%, 90%, 95% and 99% recall targets on this unchanged dataset. No tested B or C setting beat A's best setting while meeting the same target. At lower targets, A and B were close; this is a measured comparison, not a claim that A is always best.

The result has a concrete explanation: the successful B and C settings still scored the same documents as A. More selective settings either cost more or missed too many reference results. The comparison uses one document collection and one query set throughout. The comparison must keep the inputs unchanged while giving every method its own parameter choices. The practical test is whether the extra work used to avoid scores costs less than the scores it removes, at the same recall requirement and within the memory budget.

References

- [1] Exa Team. *How we built a web-scale vector database*. 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] A. Kusupati et al. *Matryoshka Representation Learning*. 2022. <https://arxiv.org/abs/2205.13147>.
- [3] Microsoft. *MS MARCO*. Passage collection and benchmark resources. <https://microsoft.github.io/msmarco/>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model documentation. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.